Question 3C

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.6.1
```

```
## Warning: package 'ggplot2' was built under R version 4.6.1
```

```
## Warning: package 'tibble' was built under R version 4.6.1
```

```
## Warning: package 'tidyr' was built under R version 4.6.1
```

```
## Warning: package 'readr' was built under R version 4.6.1
```

```
## Warning: package 'purrr' was built under R version 4.6.1
```

```
## Warning: package 'dplyr' was built under R version 4.6.1
```

```
## Warning: package 'stringr' was built under R version 4.6.1
```

```
## Warning: package 'forcats' was built under R version 4.6.1
```

```
## Warning: package 'lubridate' was built under R version 4.6.1
```

```
library(infer)
```

```
## Warning: package 'infer' was built under R version 4.6.1
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.6.1
```

```
library(rsample)
```

```
## Warning: package 'rsample' was built under R version 4.6.1
```

```
library(yardstick)
```

```
## Warning: package 'yardstick' was built under R version 4.6.1
```

```r
# Set the project root before the data-loading chunk runs
knitr::opts_knit$set(root.dir = normalizePath("../.."))
```

Data Loading

```r
load("data/biomarker-clean.RData")
set.seed(101422)
biomarker_split <- initial_split(biomarker_clean, prop = 0.8, strata = group)
train_data <- training(biomarker_split)
test_data <- testing(biomarker_split)
```

1. Keep the full ranking from each method

```r
# Rank proteins by their adjusted t-test p-values
test_fn <- function(df) {
  t_test(df, level ~ group, order = c("ASD", "TD"),
         alternative = "two-sided", var.equal = FALSE)
}

ttests_out <- train_data %>%
  select(-ados) %>%
  pivot_longer(-group, names_to = "protein", values_to = "level") %>%
  nest(data = c(level, group)) %>%
  mutate(ttest = map(data, test_fn)) %>%
  unnest(ttest) %>%
  mutate(p.adj = p.adjust(p_value, method = "BY")) %>%
  arrange(p.adj, p_value, protein)

# Fit random forest
set.seed(101422)
rf_out <- randomForest(
  x = train_data %>% select(-group, -ados),
  y = factor(train_data$group),
  ntree = 1000, importance = TRUE
)

# Break ties so each protein gets a unique rank
rank_tt <- ttests_out %>%
  transmute(protein, rank_tt = row_number())

rank_rf <- rf_out$importance %>%
  as_tibble(rownames = "protein") %>%
  arrange(desc(MeanDecreaseGini), protein) %>%
  transmute(protein, rank_rf = row_number())

ranks <- inner_join(rank_tt, rank_rf, by = "protein")
```

2. Turn each rank into a membership score

Rank 1 gets 1 and scores fall linearly to 0 after rank K.

```
K <- 30 # depth
membership <- function(r, K) pmax(0, 1 - (r - 1) / K)

fuzzy <- ranks %>%
  mutate(mu_tt = membership(rank_tt, K),
         mu_rf = membership(rank_rf, K))
```

3. Combine the scores (the fuzzy AND)

```
fuzzy <- fuzzy %>%
  mutate(
    # standard fuzzy intersection, only as strong as the weakest method
    mu_min = pmin(mu_tt, mu_rf),
    # product rule on the 0-1 scale
    mu_geo = sqrt(mu_tt * mu_rf)
  ) %>%
  arrange(desc(mu_min))

fuzzy %>% filter(mu_geo > 0) %>% head(20)
```

```
## # A tibble: 18 x 7
##    protein                rank_tt rank_rf mu_tt  mu_rf mu_min mu_geo
##    <chr>                    <int>   <int> <dbl>  <dbl>  <dbl>  <dbl>
##  1 DERM                         1       1 1      1      1      1
##  2 Calcineurin                  2       5 0.967 0.867  0.867  0.915
##  3 MAPK2                        3       7 0.933 0.8    0.8    0.864
##  4 gp130, soluble               6       8 0.833 0.767  0.767  0.799
##  5 MAPK14                       8       6 0.767 0.833  0.767  0.799
##  6 IgD                         10       2 0.7   0.967  0.7    0.823
##  7 TSP4                        15      10 0.533 0.7    0.533  0.611
##  8 Notch 1                     17       9 0.467 0.733  0.467  0.585
##  9 FSTL1                        4      19 0.9   0.4    0.4    0.6
## 10 CXCL16, soluble              5      20 0.867 0.367  0.367  0.564
## 11 Coagulation Factor IX       20      13 0.367 0.6    0.367  0.469
## 12 M2-PK                       22      17 0.3   0.467  0.3    0.374
## 13 ALCAM                       23      16 0.267 0.5    0.267  0.365
## 14 MMP-2                       25      12 0.2   0.633  0.2    0.356
## 15 TGF-b R III                 26      26 0.167 0.167  0.167  0.167
## 16 RELT                         7      27 0.8   0.133  0.133  0.327
## 17 DKK3                        18      28 0.433 0.1    0.1    0.208
## 18 CHL1                        27      30 0.133 0.0333 0.0333 0.0667
```

4. Set a threshold to get each protein panel

```
threshold <- 0.5

panels <- list(
  "Hard (top 10)" = intersect(head(ttests_out$protein, 10),
                              rank_rf %>% filter(rank_rf <= 10) %>% pull(protein)),
  "Fuzzy min" = fuzzy %>% filter(mu_min >= threshold) %>% pull(protein),
  "Fuzzy product" = fuzzy %>% filter(mu_geo >= threshold) %>% pull(protein)
)

lengths(panels)
```

```
## Hard (top 10)     Fuzzy min Fuzzy product
##             6             7            10
```

5. Fit logistic regression and evaluate on the test set

```
comparison <- tibble()
selected_proteins <- tibble()

for (rule in names(panels)) {
  panel <- panels[[rule]]

  selected_proteins <- bind_rows(selected_proteins,
                                 tibble(rule = rule, protein = panel))

  model_train <- train_data %>%
    select(group, all_of(panel)) %>%
    mutate(class = as.integer(group == "ASD")) %>%
    select(-group)

  fit <- glm(class ~ ., data = model_train, family = binomial())
  prob <- predict(fit, newdata = test_data, type = "response")
  truth <- factor(test_data$group, levels = c("TD", "ASD"))
  predicted <- factor(ifelse(prob >= 0.5, "ASD", "TD"),
                      levels = c("TD", "ASD"))

  comparison <- bind_rows(comparison, tibble(
    rule = rule,
    panel_size = length(panel),
    accuracy = accuracy_vec(truth, predicted),
    sensitivity = sens_vec(truth, predicted, event_level = "second"),
    specificity = spec_vec(truth, predicted, event_level = "second"),
    AUROC = roc_auc_vec(truth, prob, event_level = "second")
  ))
}

knitr::kable(comparison, digits = 3)
```

rule	panel_size	accuracy	sensitivity	specificity	AUROC
Hard (top 10)	6	0.688	0.562	0.812	0.715
Fuzzy min	7	0.688	0.562	0.812	0.715
Fuzzy product	10	0.656	0.562	0.750	0.680

```
knitr::kable(selected_proteins)
```

rule	protein
Hard (top 10)	DERM
Hard (top 10)	Calcineurin
Hard (top 10)	MAPK2
Hard (top 10)	gp130, soluble
Hard (top 10)	MAPK14
Hard (top 10)	IgD
Fuzzy min	DERM
Fuzzy min	Calcineurin
Fuzzy min	MAPK2
Fuzzy min	gp130, soluble
Fuzzy min	MAPK14
Fuzzy min	IgD
Fuzzy min	TSP4
Fuzzy product	DERM
Fuzzy product	Calcineurin
Fuzzy product	MAPK2
Fuzzy product	gp130, soluble
Fuzzy product	MAPK14
Fuzzy product	IgD
Fuzzy product	TSP4
Fuzzy product	Notch 1
Fuzzy product	FSTL1
Fuzzy product	CXCL16, soluble

```
# Save the results
write.csv(comparison, "results/tables/q3c_comparison.csv", row.names = FALSE)
write.csv(selected_proteins, "results/tables/q3c_selected_proteins.csv",
          row.names = FALSE)
```